

RV College of Engineering
Introduction to Python Programming Laboratory [CI115IA]
Project Based Learning Report
2025-26

Material Usage Calculator (MUC)

USN	Name
RVCE25BAI015	Medhansh Pratap Singh
RVCE25BAI072	Medha Swetha Muguda
RVCE25BAI175	Namya K M
RVCE25BAI184	Ishan Jain

Student(s)Mentor

Name, Designation & Department

1. **Introduction**

Project Name: Material Usage Calculator (MUC).

Nature: Python-based command-line application (CLI).

Purpose: Provides quick, reliable estimates for material quantities and costs for:

- Construction (Bricks, Cement, Sand).
- Household Renovation (Paint, Flooring).

Structure: Composed of four modular Python files: calc.py, check.py, data.py, and main.py.

1. **Problem Definition**

- **Challenge:** The difficulty for homeowners and small contractors in accurately estimating construction and renovation materials (bricks, cement, sand, paint, tiles).
- **Consequences of Error:** Underestimation leads to delays/multiple purchases; overestimation leads to waste and excess cost.

- **Goal:** To create a simple, accessible, and mathematically sound tool for reliable preliminary material estimates.

Background Information:

- **Manual Complexity:** Estimation requires complex calculations involving volume, unit conversions, practical ratios (e.g., cement:sand), and mandatory wastage factors.
- **Industry Practice:** The project codifies standard industry practices, such as:
 - Using empirical constants (e.g., 25% wall volume is mortar).
 - Factoring in standard wastage (e.g., 5% for bricks and tiles).
- **Calculation Examples:**
 - Cement/Sand requires calculating wall volume, then mortar volume, and finally partitioning based on the mix ratio.
 - Paint requires calculating net wall area (total area minus openings), then dividing by the specific coverage rate, and multiplying by the number of coats.

1. Objectives

Primary Objectives:

1. **Develop Core Calculation Functions:**
 - Bricks (integer count, with wastage).
 - Cement (in 50 kg bags) and Sand (in cubic feet).
 - Paint (in liters, including coats and openings).
 - Flooring Tiles (integer count, with wastage).
1. **Implement CLI Interface:** Create a user-friendly menu system and structured flows (construction_flow, household_flow).
2. **Ensure Robust Input Validation:** Use utility functions to enforce non-negative float/integer input and correct string formats.
3. **Implement Transaction History:** Record results (quantity and cost) into a persistent, timestamped CSV file (history.csv).

Secondary Objectives:

- Support both metric (meters) and imperial (feet) units for household calculations.
- Allow optional cost calculation for materials and save costs in the history.
- Maintain high code quality through modularity and clear documentation.

1. Methodology

4.1 Approach:

- **Architecture:** Modular, three-tier structure using separate files for distinct concerns:
 - calc.py: Pure mathematical logic.
 - check.py: Input validation and parsing.
 - data.py: CSV file management (persistence).
 - main.py: User interface and flow orchestration.
- **Strategy:** Prioritize logic independence; calculation functions do not depend on user input methods.

4.2 Procedures:

Step	Description	Key Modules
1-5	Calculation Logic: Define constants and develop the five core calculation functions (Bricks, Cement/Sand, Paint, Flooring).	calc.py
6	Validation Logic: Create robust input utilities (get_positive_float, yes_no, parse_tile_size).	check.py
7	Data Management: Implement CSV file creation, writing, and structured reading for history viewing.	data.py
8	Flow Integration: Build the main() function, menus, and the main user-guided flows.	main.py
9	Cost & Persistence: Integrate cost inputs and calls to data.save_transaction within the user flows.	main.py, data.py
10	Testing and Refinement.	All

5. Project Execution

5.1 Planning and Design:

- **Interface Choice:** CLI chosen to focus resources on logic accuracy and robustness over graphical development.
- **Modularity:** Design decision to keep calculation parameters (e.g., WASTAGE_FACTOR_BRICKS) as constants in calc.py for easy future modification.
- **User Flow:** Designed to be sequential and question-driven, minimizing user confusion.

5.2 Implementation:

- **Brick Formula:**
$$\text{Bricks} = \left\lceil \frac{L \times H \times T}{V_{\text{brick}}} \times 1.05 \right\rceil$$
- **Mortar Ratios:** Mortar volume (V_{mortar}) is calculated as $V_{\text{wall}} \times 0.25$. Cement and sand volumes are derived from V_{mortar} using the ratio components $\frac{C}{C+S}$ and $\frac{S}{C+S}$.
- **Unit Conversion (Paint):** The conversion factor $1 \text{ m}^2 / 10.7639 \approx 1 \text{ ft}^2$ is used to convert the default coverage rate (sqft/liter) to sqm/liter when the user inputs metric dimensions.
- **Defensive Coding:** check.py uses infinite loops (while True) and try-except blocks to guarantee valid input types and ranges before calculation.

6. Tools and Techniques Used

6.1 Tools:

- **Python 3:** Core language.

- **math Module:** Used for `math.ceil()` to ensure required materials are rounded up to the next whole unit (e.g., bricks, bags, liters).
- **csv Module:** For structured, persistent data storage (history.csv).
- **pathlib:** For robust file system interactions (`_ensure_csv_exists`).
- **datetime:** For generating transaction timestamps.

6.2 Techniques:

- **Modular Programming:** Key technique for organizing code into reusable, logical components (calc, check, data).
- **Defensive Programming:** Implemented via input validation to make the application fault-tolerant against user errors.
- **Applied Mathematics:** Using geometric formulas (volumes, areas) combined with industry-specific empirical constants (e.g., material densities, wastage factors).
- **Unit Handling:** Explicitly managing and converting units between imperial and metric systems to ensure accurate results regardless of user choice.

7. Results and Discussion

7.1 Final Results:

- **Full Functionality:** All five estimation types (Bricks, Cement, Sand, Paint, Tiles) are fully functional and yield expected, rounded-up results.
- **Accuracy:** Estimates include standard 5% wastage for solids (bricks/tiles) and use industry-standard constants for mortar composition.
- **Data Integrity:** The transaction history is correctly saved, timestamped, and displayed in a formatted table.

7.2 Discussion:

- **Objectives Met:** All primary and secondary objectives were successfully achieved.
- **Significance:** The tool provides practical, error-reduced material planning, making it valuable for student or introductory use cases.
- **Key Approximations:** The most significant approximation is the **25% mortar fraction** of the total wall volume. This may cause slight variance from actual usage but is a common, acceptable simplifying assumption.
- **Learning Point:** The necessity of careful unit handling was a major lesson, particularly the conversion of the area-based paint coverage rate.

8. Conclusion

9.1 Summary:

- **Project Outcome:** A robust, modular Material Usage Calculator CLI application was successfully developed.
- **Success Factors:** High accuracy due to the integration of applied math and industry constants, combined with strong input validation and persistent data storage.
- **Demonstration:** The project effectively demonstrates skills in software architecture, defensive coding, and the practical application of mathematical formulas to a real-world problem.

9.2 Personal Reflection:

- **Technical Gain:** Deepened understanding of **modular design principles** and the power of separating concerns between logic, UI, and data management.
- **Applied Learning:** The process of translating abstract formulas (ratios, densities, volumes) into working, robust Python code was highly challenging and rewarding, particularly in engineering the cement/sand calculation flow.
- **Impact:** The project provided practical skills in creating useful, reliable tools resistant to common user errors.